MCMP117 · WEEK 9

Fundamentals of Operating Systems

Week 9 · Linux Shell Basics & Filesystem Navigation

From theory to the terminal: where am I, what is here, and how do I move?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

The Theory Half in Four Ideas

The OS

A resource manager and an extended machine. The kernel runs in kernel mode; everything else in user mode (W1–W2).

System calls

The only door from a program into the kernel: open, read, write, fork, exec … (W2–W3).

Processes

A program in execution: PID, state, PCB, fork() + exec(), scheduled by the CPU scheduler (W3–W5).

Memory & files

Paging and virtual memory (W6–W7); files with attributes in inodes; a tree of directories with paths (W7).

The midterm tested these ideas on paper. From today you will see them at work: every command you type becomes a process, and every file you list is a node of the directory tree from Week 7.

Today's Agenda

1. **Getting a Linux terminal** Lab PCs, WSL on Windows, or a VirtualBox VM — and the lab folder.
2. **Terminal, shell, kernel** Who does what when you type a command; commands become system calls.
3. **First commands** The prompt, command anatomy, whoami, date, echo, uname.
4. **Looking around** pwd and ls; reading ls -l line by line.
5. **Moving around** The Linux directory tree (FHS), cd, absolute and relative paths.
6. **Working smarter + lab** Help, tab completion, history, shortcuts — then the treasure hunt.

PART 1

Getting a Linux Terminal

Three ways to have a Linux shell in front of you — pick one today.

Three Ways to Get a Linux Shell

1 · Lab PCs

- Already installed: log in and open Terminal.
- [Room, login and which system: to be confirmed by the instructor]
- Nothing to install — but your files stay on that PC.

2 · WSL (Windows 10/11)

- Windows Subsystem for Linux: a real Linux kernel and Ubuntu inside Windows.
- One command: wsl --install
- Recommended for Windows laptops: fast, light, no reboot to switch.

3 · VirtualBox + Ubuntu

- A complete Ubuntu PC (with desktop) running in a window.
- Needs about 4 GB RAM and 25 GB of disk for the VM.
- Works on Windows and Intel Macs; you also learn installation.

Any of the three is fine: the commands are identical. Output on your machine (dates, sizes, user names, PIDs) will differ from the slides.

WSL: Ubuntu Inside Windows in Four Steps

```
# 1. PowerShell "Run as administrator"
PS C:\Windows\system32> wsl --install
# ... downloads WSL and Ubuntu, then asks
#     you to restart Windows

# after the restart: check what is installed
PS C:\Users\ayse> wsl -l -v
```

Requirements and problems

- Windows 10 version 2004 (build 19041) or newer, or Windows 11.
- Error about virtualization? Enable it in the BIOS/UEFI (Intel VT-x / AMD-V).

Step by step

- **1 · Install:** right-click Start → Terminal (Admin) or PowerShell (Admin); type wsl --install.
- **2 · Restart** Windows when asked.
- **3 · First start:** open "Ubuntu" from the Start menu; choose a Linux user name and password. The password is not shown while you type — that is normal.
- **4 · Use it:** open Ubuntu (or type wsl) whenever you need the shell.
- **Tip:** your Windows C: drive is /mnt/c inside Linux.

VirtualBox + Ubuntu: a Whole Linux PC in a Window

1

Download

VirtualBox (virtualbox.org) and the Ubuntu Desktop 24.04 LTS ISO image (ubuntu.com/download/desktop).

2

Create the VM

New → name "Ubuntu", select the ISO, type Linux / Ubuntu (64-bit).

3

Resources

Memory 4096 MB, 2 CPUs, virtual disk 25 GB (dynamically allocated).

4

Install

Start the VM, follow "Install Ubuntu". "Erase disk" erases only the virtual disk file.

5

Open a terminal

Log in, press Ctrl+Alt+T (or Activities → Terminal).

A virtual machine is the OS idea of virtualization applied to a whole computer: VirtualBox (a hypervisor) gives Ubuntu virtual CPUs, memory and a disk, so Ubuntu believes it owns the machine. Anything you break inside the VM cannot harm Windows — a perfect place to practise.

Run the Lab Setup Script Once

```
student@okan-lab:~$ pwd
/home/student
student@okan-lab:~$ ls
lab  mcmp117_lab_setup.sh
student@okan-lab:~$ bash mcmp117_lab_setup.sh
Lab folder ready: /home/student/lab
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~$ ls
lab  mcmp117_lab_setup.sh
student@okan-lab:~$ ls lab
README.txt  backup  data  docs  logs  photos  projects  scripts  src
```

Getting the script

- Download mcmp117_lab_setup.sh from the course Telegram channel [link: to be confirmed].
- Put it in your home directory, or run it from where it is (it always builds ~/lab).
- WSL: Windows Downloads = /mnt/c/Users/<you>/Downloads

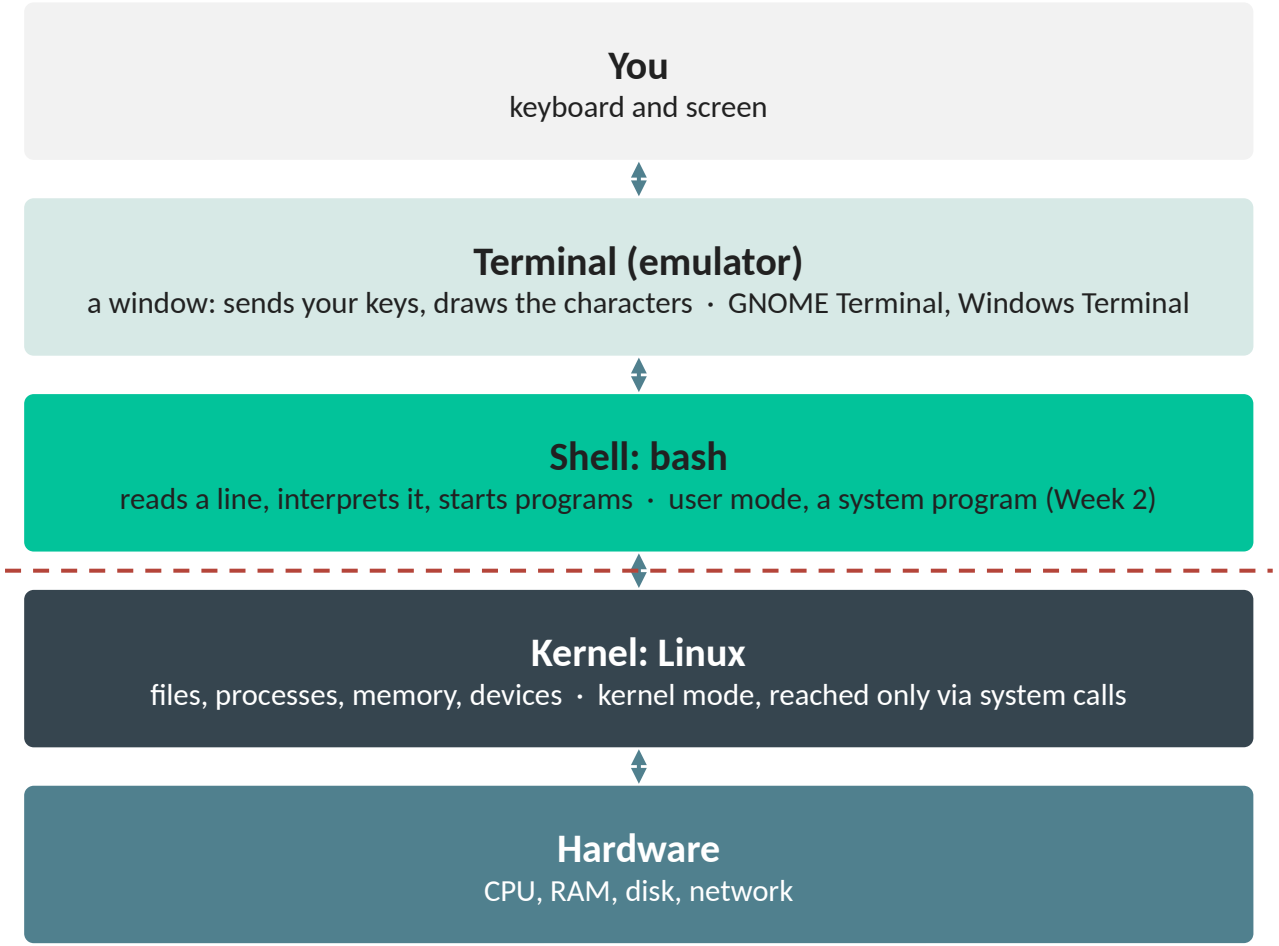
```
# run it straight from the download folder (VM / Ubuntu):
student@okan-lab:~$ bash ~/Downloads/mcmp117_lab_setup.sh
# ... or from the Windows Downloads folder (WSL):
student@okan-lab:~$ bash /mnt/c/Users/ayse/Downloads/mcmp117_lab_setup.sh
```

PART 2

Terminal, Shell, Kernel

Three different programs that are often confused.

Terminal ≠ Shell ≠ Kernel



Keep them apart

- **Terminal:** only input and output. It does not understand commands.
- **Shell:** the command interpreter (Week 3). Linux has several: bash (default on Ubuntu), zsh, sh, fish.
- **Kernel:** does the real work for every command, through system calls (red dashed line = system-call boundary).
- **"Linux"** strictly means the kernel; Ubuntu = Linux kernel + GNU tools + much more (Week 2).

LINK TO WEEK 3

What Happens When You Type ls and Press Enter



The five steps

- **1 Read:** bash reads the line "ls" from the terminal.
- **2 Find:** ls is not built in, so bash searches the directories in $PATH and finds /usr/bin/ls.
- **3 fork():** bash creates a child process, a copy of itself.
- **4 exec():** the child replaces its program with /usr/bin/ls; ls runs, prints, exits.
- **5 wait():** the parent shell waits, then prints the next prompt.

This is exactly the fork–exec–wait pattern of Week 3. Every external command you type is a new process with its own PID.

LINK TO WEEK 2

Commands Become System Calls: strace ls

```
student@okan-lab:~/lab$ strace -e trace=execve,getdents64,write ls docs > /dev/null
execve("/usr/bin/ls", ["ls", "docs"], 0x7ffc5b53bce8 /* 13 vars */) = 0
getdents64(3, 0x55fd38daa8e0 /* 9 entries */, 32768) = 288
getdents64(3, 0x55fd38daa8e0 /* 0 entries */, 32768) = 0
write(1, "final report.txt\nlinux_quotes.tx"..., 86) = 86
+++ exited with 0 +++
```

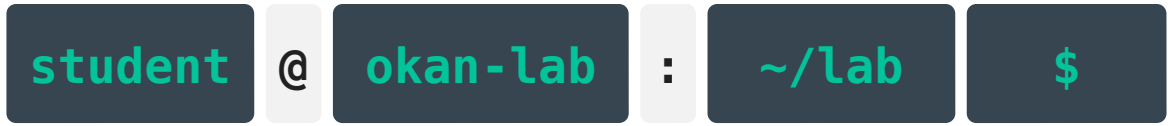
Reading the trace

- **execve:** the child turns into /usr/bin/ls with arguments ["ls", "docs"].
- **getdents64:** "get directory entries" — reads the directory: 9 entries (6 files, .settings, . and ..).
- **write(1, ...):** prints 86 bytes to file descriptor 1 (standard output).

Why this matters

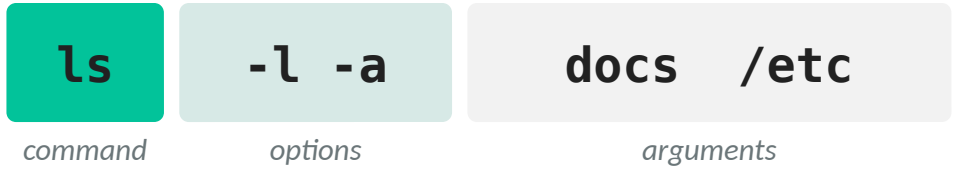
- ls cannot read the disk itself: it asks the kernel, as every program must (dual mode, Week 2).
- A directory is a file holding (name, inode) pairs (Week 7); getdents64 reads those pairs.
- > /dev/null hides ls's own output (Week 12), so only strace's report is shown.

The Prompt and the Anatomy of a Command



~ = your home directory /home/student · # instead of $ = root (the administrator)

command -options arguments



separated by spaces · case matters: ls ≠ LS

Example	Command	Options	Arguments
pwd	pwd	—	—
ls -la docs	ls	-l -a (combined)	docs
ls --all docs	ls	--all (long form)	docs
cd ../src	cd	—	../src
uname -r	uname	-r	—
head -n 4 file	head	-n with value 4	file

Options change HOW a command works (usually start with - or --); arguments say WHAT it works on (files, directories, text).

LIVE DEMO

First Commands: Who, Where, When, What

```
student@okan-lab:~$ whoami
student
student@okan-lab:~$ hostname
okan-lab
student@okan-lab:~$ date
Mon Oct 12 08:49:13 UTC 2026
student@okan-lab:~$ echo Hello from the shell
Hello from the shell
student@okan-lab:~$ echo My shell is $SHELL and my home is ~
My shell is /bin/bash and my home is /home/student
student@okan-lab:~$ uname
Linux
student@okan-lab:~$ uname -r
6.18.44-fc-v114
student@okan-lab:~$ uname -a
Linux okan-lab 6.18.44-fc-v114 #1 SMP PREEMPT_DYNAMIC @0 x86_64 x86_64 x86_64 GNU/Linux
```

whoami: your user name · hostname: the computer's name · date: date and time · echo: prints its arguments (the shell replaced $SHELL and ~ before echo ran) · uname -r / -a: kernel version / everything · clear or Ctrl+L: clear the screen

PART 3

Where Am I? What Is Here?

pwd prints your position in the tree; ls lists what is there.

LIVE DEMO

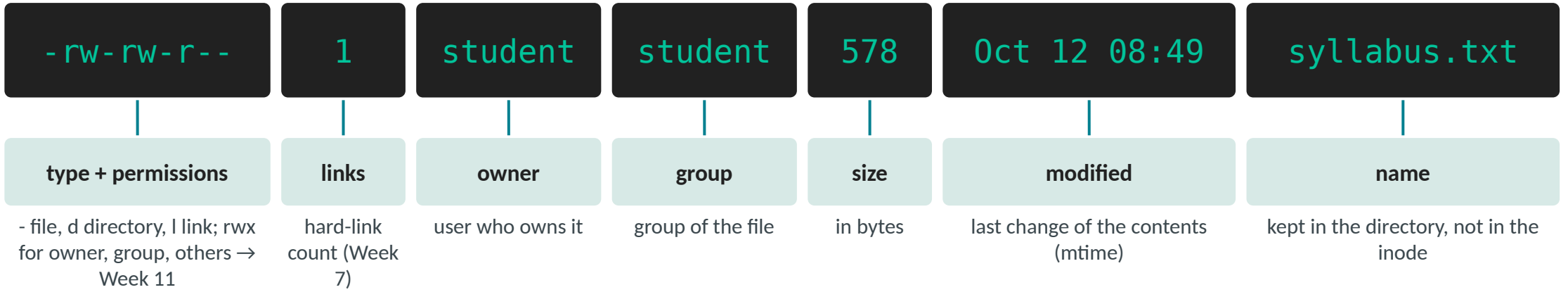
pwd and ls in the Lab Folder

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~/lab$ ls -F
README.txt  backup/  data/  docs/  logs/  photos/  projects/  scripts/  src/
student@okan-lab:~/lab$ ls docs
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ ls -a docs
 .    .settings           linux_quotes.txt   notes_02.txt   syllabus.txt
 ..  'final report.txt'   notes_01.txt       notes_03.txt
```

What to observe

- **pwd** = print working directory: an absolute path.
- **ls** lists the current directory; ls docs lists another one.
- **-F** marks directories with /.
- **-a** shows hidden names starting with a dot: .settings, . and ..
- **Quotes:** ls shows 'final report.txt' in quotes because the name has a space.

Reading One Line of ls -l



Directories look like this

- drwxrwxr-x 4 student student 4096 … projects
- d = directory; size 4096 = one block of entries; link count 4 = 2 + its 2 subdirectories.

Almost all of it is the inode

- Type, permissions, links, owner, group, size and times are inode fields (Week 7).
- Only the name comes from the directory entry. stat shows the full inode (Week 10).

LIVE DEMO

ls -l and ls -lh: Sizes Humans Can Read

```
student@okan-lab:~/lab$ ls -l docs
total 24
-rw-rw-r-- 1 student student  53 Oct 12 08:49 'final report.txt'
-rw-rw-r-- 1 student student 302 Oct 12 08:49  linux_quotes.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_01.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_02.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_03.txt
-rw-rw-r-- 1 student student 578 Oct 12 08:49  syllabus.txt
student@okan-lab:~/lab$ ls -lh logs data
data:
total 12K
-rw-rw-r-- 1 student student  227 Oct 12 08:49 cities.txt
-rw-rw-r-- 1 student student   80 Oct 12 08:49 numbers.txt
-rw-rw-r-- 1 student student 1.2K Oct 12 08:49 students.csv

logs:
total 32K
-rw-rw-r-- 1 student student  24K Oct 12 08:49 access.log
-rw-rw-r-- 1 student student 5.1K Oct 12 08:49 system.log
```

-h = human-readable

- K, M, G are powers of 1024: 1K = 2^{10} bytes.
- access.log: 23,604 B ÷ 1024 ≈ 23.05 → shown as 24K (ls rounds up).
- system.log: 5,207 B ÷ 1024 ≈ 5.09 → 5.1K.

The "total" line

- Disk space used by the listed files, in 1K blocks. Each small file takes a whole 4K block: 6 × 4 = 24.

LIVE DEMO

Whole Subtrees: ls -R and tree

```
student@okan-lab:~/lab$ ls -R projects
projects:
alpha  beta

projects/alpha:
README.md  docs  src

projects/alpha/docs:
design.txt

projects/alpha/src:
main.c

projects/beta:
README.md  docs  src

projects/beta/docs:
design.txt

projects/beta/src:
main.c
```

```
student@okan-lab:~/lab$ tree projects
projects
├── alpha
│   ├── README.md
│   ├── docs
│   │   └── design.txt
│   └── src
│       └── main.c
└── beta
    ├── README.md
    ├── docs
    │   └── design.txt
    └── src
        └── main.c

7 directories, 6 files
```

Compare

- **ls -R:** recursive; one block per directory.
- **tree:** the same as a picture — the Week 7 tree.
- Install: sudo apt install tree
- Careful: ls -R / lists the whole disk. Ctrl+C stops it.

CHEAT SHEET

ls Options You Will Use All Semester

Option	Meaning	Example
-l	long listing: type, permissions, links, owner, group, size, time, name	ls -l docs
-a / -A	all, including hidden dot-files (-A: without . and ..)	ls -a ~
-h	human-readable sizes (with -l)	ls -lh logs
-R	recursive: also list every subdirectory	ls -R projects
-F	append / to directories (* to executables, @ to links)	ls -F
-d	the directory itself, not its contents	ls -ld projects
-S / -t / -r	sort by size / by time (newest first) / reverse the order	ls -lS docs
-1 / -i	one name per line / show inode numbers (Week 7)	ls -1 photos

INTERMISSION

10-Minute Break

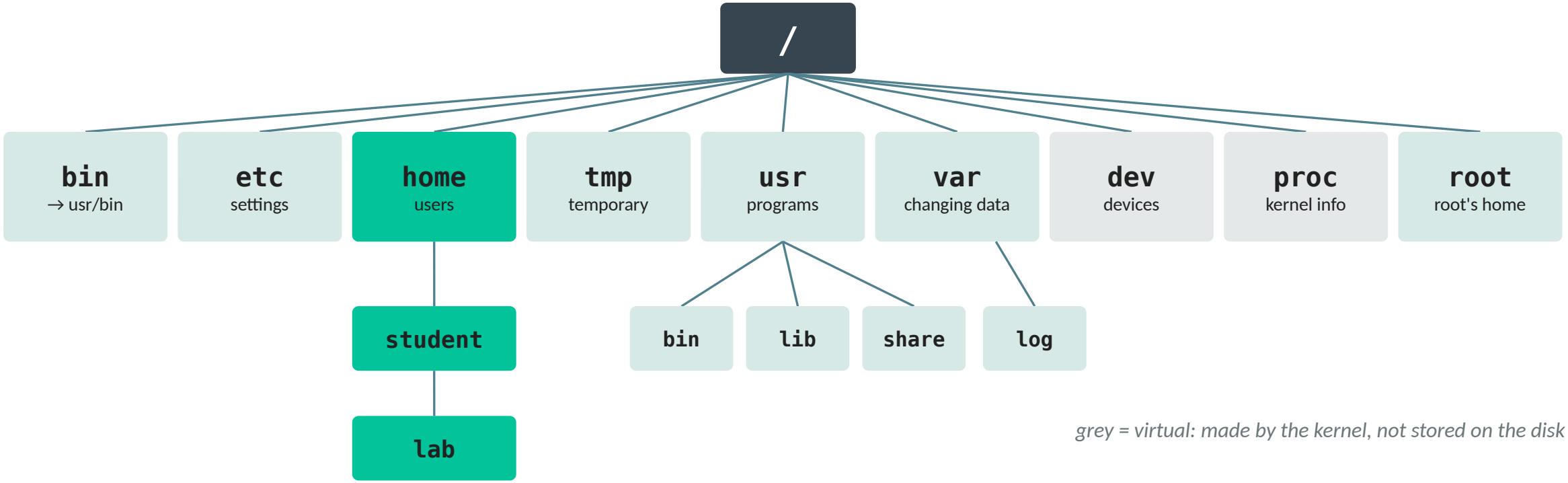
Next: the Linux directory tree, cd, paths, getting help — then the lab.

PART 4

Moving Around the Tree

One tree, one root: /. No drive letters.

The Linux Directory Tree



grey = virtual: made by the kernel, not stored on the disk

/home/student/lab = ~/lab

Everything — disks, USB sticks, even devices and kernel data — appears somewhere under the single root /. Windows has one tree per drive (C:\, D:\).

LIVE DEMO

A Tour of the Tree: /, /bin, /etc, /var, /tmp

```
student@okan-lab:~$ type ls
ls is /usr/bin/ls
student@okan-lab:~$ cd /
student@okan-lab:/$ ls
bin                etc                lib64       opt   sbin                tmp
bin.usr-is-merged  home               lost+found  proc  sbin.usr-is-merged  usr
boot               lib                media       root  srv                 var
dev                lib.usr-is-merged  mnt         run   sys
student@okan-lab:/$ ls -ld /bin /sbin /lib
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /lib -> usr/lib
lrwxrwxrwx 1 root root 8 Apr 22  2024 /sbin -> usr/sbin
student@okan-lab:/$ ls -l /usr/bin/ls
-rwxr-xr-x 1 root root 142312 Jan 23  2026 /usr/bin/ls
student@okan-lab:/$ head -n 4 /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
student@okan-lab:/$ ls /var
backups  cache  lib  local  lock  log  mail  opt  run  spool  tmp
student@okan-lab:/$ ls -ld /tmp
drwxrwxrwt 16 root root 4096 Oct 12 08:49 /tmp
```

What to observe

- **/bin -> usr/bin:** a symbolic link (l); programs really live in /usr/bin.
- **/usr/bin/ls:** the ls program is just a file (142,312 bytes).
- **/etc:** text configuration files — os-release names the distribution.
- **/var:** data that grows: log, cache, spool.
- **/tmp:** everyone may write (the t: sticky bit, Week 11).

/dev and /proc: Devices and the Kernel as Files

```
student@okan-lab:/$ ls -l /dev/null /dev/tty /dev/zero
crw-rw-rw- 1 root root 1, 3 Oct  9 23:24 /dev/null
crw-rw-rw- 1 root root 5, 0 Oct  9 23:24 /dev/tty
crw-rw-rw- 1 root root 1, 5 Oct  9 23:24 /dev/zero
student@okan-lab:/$ head -n 3 /proc/meminfo
MemTotal:        8222320 kB
MemFree:         7069620 kB
MemAvailable:    7539020 kB
student@okan-lab:/$ echo $$
18432
student@okan-lab:/$ head -n 7 /proc/$$/status
Name:   bash
Umask:  0002
State:  S (sleeping)
Tgid:   18432
Ngid:   0
Pid:    18432
PPid:   18377
```

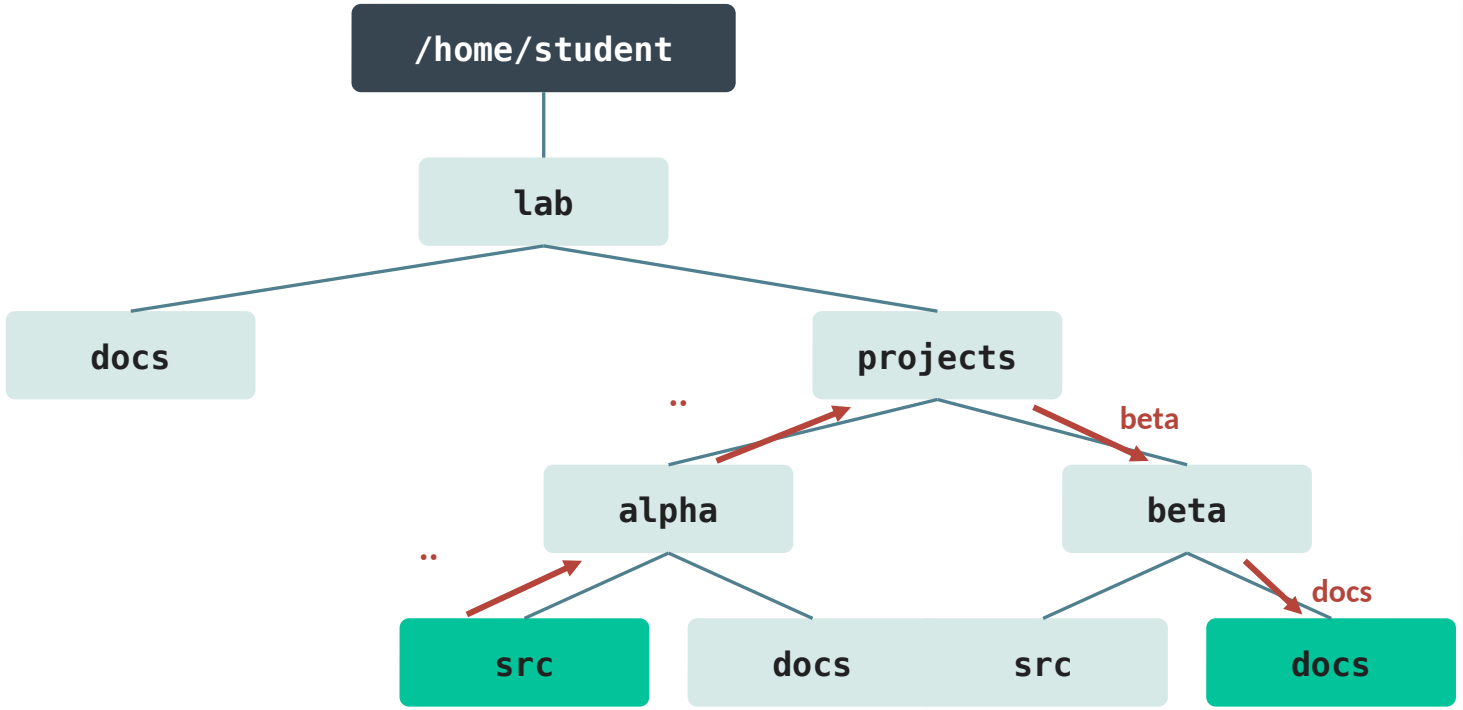
/dev: device files

- c = character device; "1, 3" are the driver numbers, not a size.
- /dev/null swallows everything written to it (Week 12).

/proc: a window into the kernel

- Not on disk: the kernel writes the text when you read it.
- $$ = PID of this shell. /proc/<PID>/status shows part of its PCB: name, state S (sleeping), PID, parent PID (Week 3).

Absolute and Relative Paths in ~/lab



from projects/alpha/src to projects/beta/docs: ../../beta/docs

Same idea as Week 7: an absolute path walks from the root, a relative path from the current directory.

Absolute

- Starts with / (or ~).
- Same meaning from anywhere.
- /home/student/lab/projects/beta/docs

Relative

- No leading /: starts at the current directory.
- .. = one level up.
- Short, but depends on where you are.

LIVE DEMO

cd: Change Directory

```
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ pwd
/home/student/lab/projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ..
student@okan-lab:~/lab/projects/alpha$ pwd
/home/student/lab/projects/alpha
student@okan-lab:~/lab/projects/alpha$ cd ../../docs
student@okan-lab:~/lab/docs$ pwd
/home/student/lab/docs
student@okan-lab:~/lab/docs$ cd /home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ cd -
/home/student/lab/docs
student@okan-lab:~/lab/docs$ cd -
/home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ cd
student@okan-lab:~$ pwd
/home/student
student@okan-lab:~$ cd /
student@okan-lab:/$ cd ~/lab/projects/alpha/docs
student@okan-lab:~/lab/projects/alpha/docs$ cd ../../beta/src
student@okan-lab:~/lab/projects/beta/src$ pwd
/home/student/lab/projects/beta/src
```

What to observe

- The prompt shows where you are after each cd.
- cd .. goes one level up; ../../docs goes up twice, then down.
- An absolute path works from anywhere.
- cd - jumps back to the previous directory and prints it.
- cd alone takes you home; cd / to the root.
- cd prints nothing when it succeeds.

SHORTCUTS

Five Special Names for Directories

Name	Meaning	Example	Result (from ~/lab/projects/alpha)
/	the root of the whole tree	cd /	/
~	your home directory	cd ~/lab	/home/student/lab
.	the current directory	ls .	lists alpha (same as ls)
..	the parent directory	cd ../beta	/home/student/lab/projects/beta
-	the previous directory (cd only)	cd -	back to where you were before

cd with no argument = cd ~. The ".." of / is / itself: you cannot go above the root.

Why cd Must Be Built into the Shell

```
student@okan-lab:~/lab$ /bin/pwd
/home/student/lab
student@okan-lab:~/lab$ strace -e trace=getcwd /bin/pwd
getcwd("/home/student/lab", 4096)       = 18
/home/student/lab
+++ exited with 0 +++
student@okan-lab:~/lab$ strace -e trace=chdir bash -c 'cd /tmp'
chdir("/tmp")                           = 0
+++ exited with 0 +++
student@okan-lab:~/lab$ pwd
/home/student/lab
```

The experiment

- /bin/pwd is a separate program: it asks the kernel with getcwd().
- A child bash runs cd /tmp: the kernel's chdir("/tmp") succeeds …
- … but our shell is still in ~/lab.

The current directory is part of each process's state (kept by the kernel next to the PCB, Week 3). A child can change only its own directory, never its parent's. If cd were a program in /usr/bin, it would change the directory of a child process that exits immediately — and nothing would happen. So cd is a builtin: bash calls chdir() on itself. type cd confirms it.

PART 5

Working Smarter

Nobody remembers every option: know where to look, and let the shell type for you.

Four Ways to Get Help

```
student@okan-lab:~$ ls --help | head -n 12
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  do not ignore entries starting with .
  -A, --almost-all           do not list implied . and ..
...
student@okan-lab:~$ type cd pwd ls
cd is a shell builtin
pwd is a shell builtin
ls is /usr/bin/ls
student@okan-lab:~$ help cd | head -n 5
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.

    Change the current directory to DIR.  The default DIR is the value of the
    HOME shell variable. If DIR is "-", it is converted to $OLDPWD.
```

- **cmd --help:** short summary of the options (most programs).
- **man cmd:** the full manual page — sections NAME, SYNOPSIS, DESCRIPTION. Space = next page, /word = search, q = quit.
- **help cmd:** for shell builtins (cd, pwd, history) — they have no program file.
- **type cmd:** builtin or program file?
- **whatis / apropos:** one-line description / search the manuals by keyword: apropos directory.
- **tldr cmd** (optional, sudo apt install tldr): short examples.

LIVE DEMO

Tab Completion, History and Keyboard Shortcuts

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ cd projects/alpha/
student@okan-lab:~/lab/projects/alpha$ pwd
/home/student/lab/projects/alpha
student@okan-lab:~/lab/projects/alpha$ cd
student@okan-lab:~$ ls ~/lab/docs/syllabus.txt
/home/student/lab/docs/syllabus.txt
student@okan-lab:~$ whoami
student
student@okan-lab:~$ !!
whoami
student
student@okan-lab:~$ history
    1  cd ~/lab
    2  cd projects/alpha/
    3  pwd
    4  cd
    5  ls ~/lab/docs/syllabus.txt
    6  whoami
    7  whoami
    8  history
student@okan-lab:~$ !5
ls ~/lab/docs/syllabus.txt
/home/student/lab/docs/syllabus.txt
```

Key / command	What it does
Tab	complete a command, file or directory name
Tab Tab	list all possible completions
↑ / ↓	previous / next command from history
history	numbered list of your commands
!! · !n	repeat the last command · command number n
Ctrl+C	interrupt (stop) the running command
Ctrl+L	clear the screen (like clear)
Ctrl+A / Ctrl+E	jump to start / end of the line

CHEAT SHEET

Week 9 Command Reference

Command	Purpose	Command	Purpose
whoami	print your user name	pwd	print the working directory
hostname	print the computer's name	ls [-laRhF] [dir]	list a directory
date	print date and time	cd [dir]	change directory (~ . .. - /)
echo text	print text	tree [dir]	draw a directory tree
uname -a	kernel name, version, machine	cat file	print a file (Week 10)
clear	clear the screen (Ctrl+L)	type cmd	builtin or program?
history	list previous commands	cmd --help	short help
!! !n	repeat a command	man cmd · help cmd	manual · builtin help

IN-CLASS LAB · 30–40 MINUTES

Lab Part A: Treasure Hunt in ~/lab

Follow the clues — use only cd, ls, pwd, cat

- **Start:** re-create the lab (bash mcmp117_lab_setup.sh), then cd ~/lab.
- **Clue 1:** read README.txt. Which directory holds "a small C program"? Go there.
- **Clue 2:** list it and read the C file. What does the program print?
- **Clue 3:** with ONE relative cd, move to the directory of short text documents. Write the command.
- **Clue 4:** one file there has a space in its name. Read it: its last line is the password.
- **Clue 5:** a hidden file is there too. Which editor does it set?
- **Finish:** cd to projects/beta/src with one relative cd, run pwd.

Hand in (on paper or Telegram)

- The command you used for each clue.
- The answers to clues 2, 4 and 5.
- The final pwd output.
- Names with spaces need quotes: cat "final report.txt"
- Stuck? ls -a, pwd and Tab are your friends.

IN-CLASS LAB

Lab Part B: Look, Count, Explain

B1 · Count

In ~/lab: how many entries does ls show? How many are directories? (Use ls -F or ls -l.)

B2 · Biggest file

Which file in docs is the largest, and how many bytes? Confirm with ls -lS docs.

B3 · Human sizes

ls -l logs and ls -lh logs: write both sizes of access.log and explain the K.

B4 · Predict

From ~/lab/projects: cd alpha; cd ../beta/src; cd ..; cd - Where are you now? Predict, then check.

B5 · The system

Where is the program ls? What is /bin really? Which Ubuntu version is installed?

B6 · Challenge

ls -ld ~/lab shows link count 10, docs shows 2. Explain both numbers.

The full Lab Sheet (18 graded exercises, 100 points) and its solutions are in this week's course notes.

Error Messages and What They Mean

```
student@okan-lab:~/lab$ LS
bash: LS: command not found
student@okan-lab:~/lab$ ls -z
ls: invalid option -- 'z'
Try 'ls --help' for more information.
student@okan-lab:~/lab$ ls Docs
ls: cannot access 'Docs': No such file or directory
student@okan-lab:~/lab$ cd nowhere
bash: cd: nowhere: No such file or directory
student@okan-lab:~/lab$ cd README.txt
bash: cd: README.txt: Not a directory
student@okan-lab:~/lab$ ls docs/final report.txt
ls: cannot access 'docs/final': No such file or directory
ls: cannot access 'report.txt': No such file or directory
student@okan-lab:~/lab$ ls "docs/final report.txt"
'docs/final report.txt'
student@okan-lab:~/lab$ cd docs/final report.txt
bash: cd: too many arguments
student@okan-lab:~/lab$ cd /root
bash: cd: /root: Permission denied
```

- **command not found:** typo, wrong case (LS), or the program is not installed.
- **invalid option:** check cmd --help.
- **No such file or directory:** wrong name/case, or you are in a different directory — run pwd and ls.
- **Not a directory:** cd works on directories, not files.
- **Space in a name:** the shell splits it into two arguments → quote it "..." or escape it \ .
- **Permission denied:** not your directory (/root) — Week 11.

QUICK QUIZ

Check Your Understanding

1. You are in /home/student/lab/projects/alpha. Where are you after cd ../beta/src? And after cd - ?

2. In ls -lh /var/log, which part is the command, which are options and which is the argument?

3. A file has 23,604 bytes. What size does ls -lh show, and why?

4. Why is cd a shell builtin and not a program in /usr/bin? Which Week 3 idea explains it?

Week 9 — Key Takeaways

- **Getting Linux:** lab PCs, WSL (wsl --install) or a VirtualBox VM; ~/lab is created by mcmp117_lab_setup.sh.
- **Terminal ≠ shell ≠ kernel:** the terminal shows text, bash interprets commands, the kernel does the work via system calls.
- **Commands are processes:** bash forks, the child execs /usr/bin/ls, the shell waits (Week 3); builtins like cd run inside bash.
- **Anatomy:** command -options arguments; spaces separate words; case matters.
- **Looking around:** pwd, ls -l -a -h -R; ls -l shows the inode fields of Week 7.
- **Moving:** one tree from /; absolute vs relative paths; ~ . .. - with cd.
- **Working smarter:** --help, man, help, type; Tab, history, !!, Ctrl+C, Ctrl+L.

NEXT WEEK

File & Directory Operations

Week 10 will cover:

- mkdir, touch, cp, mv, rm, rmdir — and why rm has no recycle bin
- Wildcards * ? [ ] and brace expansion { } — expanded by the shell
- Viewing files: cat, less, head, tail, wc, file, stat, du, df
- Hard and symbolic links with ln and ln -s (inodes from Week 7)
- Lab: organise photos/ and docs/ with wildcards, make backups

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz